



Functions, Scope & Type Hints

Python Mastery — Part 1: Foundations · Week 5

Rathinam Trainers & Consultants

Live on Microsoft Teams · recorded

Recap & today

The turning point of the course

Last week — Control Flow

- ▶ Branching with `if` / `elif` / `else` and **truthiness**
- ▶ Looping with `for`, `while`, and `range()`
- ▶ Steering: `break`, `continue`, `pass`, loop-`else`
- ▶ Dispatching on shape with `match` / `case`

Your code can decide and repeat. Today it learns to be **reused**.

Today's goal

By the end of this session you'll understand how to:

- ▶ Define functions with `def`, call them, and `return` values
- ▶ Write **any** signature: positional, keyword, default, `*args`, `**kwargs`, keyword-only, positional-only
- ▶ Explain **scope** with **LEGB**, and use `global` / `nonlocal`
- ▶ Write **lambdas**, and annotate code with **type hints**

The unit of reuse

`def`, `return`, `docstrings`

Defining & calling

```
def greet(name):  
    return f"Hi, {name}!"  
  
message = greet("Asha")    # -> 'Hi, Asha!'
```

- ▶ **def** names it · the **body** is indented, like a loop
- ▶ **Parameters** are the names in the **def** line
- ▶ **Arguments** are the values you pass at the call
- ▶ Defining ≠ running — the body runs only when **called**

return and the invisible None

```
def add(a, b):  
    return a + b          # hands a value back  
  
def shout(text):  
    print(text.upper())   # prints, returns nothing
```

```
>>> repr(shout("hi"))  
'None'
```

- ▶ No `return`? You still get `None` back
- ▶ `print` **shows**; `return` **hands back** — very different

Docstrings & `help()`

```
def greet(name: str, greeting: str = "Hi") -> str:
    """Return a friendly greeting."""
    return f"{greeting}, {name}!"
```

```
>>> help(greet)
Help on function greet in module __main__:

greet(name: str, greeting: str = 'Hi') -> str
    Return a friendly greeting.
```

A **docstring** is the first string in the body — `help()` reads it.

Let's write every kind of signature.

Watch this — Demo 9

The argument-grammar tour

- ▶ `def greet(name, greeting="Hi")` — positional, keyword, default
- ▶ Add `*args` and `**kwargs` — see what they capture
- ▶ **Keyword-only** after `*` · **positional-only** before `/`
- ▶ The **mutable-default trap** — and the `None` fix

Positional, keyword, default

```
>>> def greet(name, greeting="Hi"):
...     return f"{greeting}, {name}!"
>>> greet("Asha") # positional
'Hi, Asha!'
>>> greet("Asha", "Hello") # override default
'Hello, Asha!'
>>> greet(greeting="Welcome", name="Ravi") # keyword – any order
'Welcome, Ravi!'
>>> greet()
TypeError: greet() missing 1 required positional argument: 'name'
```

`*args` and `**kwargs`

```
>>> def report(*args, **kwargs):  
...     return f"args={args} kwargs={kwargs}"  
>>> report(1, 2, 3)  
'args=(1, 2, 3) kwargs={}'  
>>> report(1, 2, mode="fast", debug=True)  
"args=(1, 2) kwargs={'mode': 'fast', 'debug': True}"
```

- ▶ `*args` → leftover **positionals** as a **tuple**
- ▶ `**kwargs` → leftover **keywords** as a **dict**
- ▶ The `*` and `**` are the syntax; the names are convention

Keyword-only & positional-only

```
>>> def make_task(title, *, priority="normal"):
...     return f"{title!r} priority={priority}"
>>> make_task("Buy milk", "high")
TypeError: make_task() takes 1 positional argument but 2 were given
```

```
>>> def power(base, exp, /):
...     return base ** exp
>>> power(base=2, exp=10)
TypeError: power() got some positional-only arguments
        passed as keyword arguments: 'base, exp'
```

Everything **after** `*` must be named · everything **before** `/` must not.

The mutable-default trap

```
>>> def add_item(item, basket=[]):  
...     basket.append(item)  
...     return basket  
>>> add_item("a")  
['a']  
>>> add_item("b")  
['a', 'b']           # ← where did "a" come from?!  
>>> add_item("c")  
['a', 'b', 'c']
```

Defaults are evaluated **once, at `def` time** — that one list is **shared forever**.

The fix — and what Ruff says

```
def add_item(item, basket=None):  
    if basket is None:  
        basket = []          # a fresh list per call  
    basket.append(item)  
    return basket
```

```
$ ruff check --select B006 trap.py  
B006 Do not use mutable data structures for argument defaults  
--> trap.py:1:27  
help: Replace with `None`; initialize within function
```

B006 is **not** in Ruff's default rules — you **opt in**.

What you just saw

- ▶ One function read **four** ways: positional, keyword, default, error
- ▶ `*args` / `**kwargs` soak up **whatever's left**
- ▶ `*` forces **names**; `/` forbids them
- ▶ A mutable default is **created once** and shared — use `None`
- ▶ Ruff catches it, **but only if you ask for** `B006`

Where names live

Scope, LEGB, and closures

Local vs global

```
x = "global x"

def show():
    x = "local x"      # a NEW name, inside this call
    return x
```

```
>>> show()
'local x'
>>> x
'global x'           # untouched
```

Assigning inside a function makes a **local** name by default.

LEGB — the four-step search

When Python meets a name, it looks in **this order** and stops at the first hit:

Step	Scope	Example
L	Local	names assigned in <i>this</i> function
E	Enclosing	the outer function wrapping this one
G	Global	names at the top of the module
B	Built-in	<code>len</code> , <code>print</code> , <code>range</code> , ...

Not found in any of the four? → `NameError`

`global` — rare, and loud

```
debug = False

def enable_debug():
    global debug          # "assign to the MODULE-level name"
    debug = True
```

- ▶ Without `global`, `debug = True` would make a **local** and vanish
- ▶ It works — but any caller can change your module's state
- ▶ **Prefer returning a value.** Reach for `global` rarely and deliberately

`nonlocal` — the counter closure

```
def make_counter():  
    count = 0  
  
    def increment():  
        nonlocal count      # the ENCLOSING count, not a new one  
        count += 1  
        return count  
  
    return increment
```

```
>>> c = make_counter()  
>>> c(); c(); c()  
1  
2  
3
```

Lambdas — one-expression functions

```
double = lambda n: n * 2          # same as a tiny def
```

```
>>> tasks = [("write report", 1), ("buy milk", 3), ("call bank", 2)]
>>> sorted(tasks, key=lambda t: t[1])
[('write report', 1), ('call bank', 2), ('buy milk', 3)]
```

- ▶ **One expression**, no statements, result returned automatically
- ▶ Real home: a throwaway `key=` for `sorted()` / `min()` / `max()`
- ▶ Needs a name or a docstring? Use `def` instead

Type hints

Telling the editor what you mean

Annotating parameters & returns

```
def add(a: int, b: int) -> int:  
    return a + b
```

- ▶ `a: int` — "this parameter is meant to be an `int`"
- ▶ `-> int` — "this function is meant to hand back an `int`"
- ▶ **Documentation your editor can read** — and act on
- ▶ Optional: add them where they earn their keep

Modern hint syntax

```
def summarise(  
    titles: list[str],  
    counts: dict[str, int],  
    note: str | None = None,  
) -> str:  
    ...
```

- ▶ `list[str]`, `dict[str, int]` — built-in generics, **no import**
- ▶ `str | None` — "a string **or** nothing" (the old `Optional[str]`)
- ▶ Skip `typing.List` / `typing.Optional` — the built-ins are current

Hints are **not** enforced

```
>>> def add(a: int, b: int) -> int:
...     return a + b
>>> add("two", "three")
'twothree'                                # no error. none at all.
```

- ▶ Python **ignores** hints at runtime — they are not checks
- ▶ The payoff is **before** you run: editor, autocomplete, type checkers
- ▶ Part 2 turns these into real, enforced checks

Let's watch names resolve.

Watch this — Demo 10

Scope / LEGB + type hints in VS Code

- ▶ A name resolving $L \rightarrow E \rightarrow G \rightarrow B$, live
- ▶ A `counter()` that works only because of `nonlocal`
- ▶ A `global` toggle
- ▶ **Pylance** flagging a wrong-typed call — and autocompleting a typed param
- ▶ Proof the **runtime doesn't care**

LEGB, live

```
>>> name = "global name"
>>> def outer():
...     name = "enclosing name"
...     def inner():
...         return name          # no local -> look Enclosing
...     return inner()
>>> outer()
'enclosing name'
```

```
>>> def broken():
...     count = count + 1        # count is local... and unset
>>> broken()
UnboundLocalError: cannot access local variable 'count'
where it is not associated with a value
```

The `nonlocal` counter

```
>>> c = make_counter()
>>> c()
1
>>> c()
2
>>> c2 = make_counter()      # a fresh, independent count
>>> c2()
1
```

Remove `nonlocal` and it breaks:

```
UnboundLocalError: cannot access local variable 'count'
where it is not associated with a value
```

Each closure keeps **its own** enclosing `count` alive.

Pylance catches it

```
def greet(name: str, greeting: str = "Hi") -> str:  
    return f"{greeting}, {name}!"  
  
greet(123, "Hello")      # ← red squiggle under 123
```

Argument of type `"Literal[123]"` cannot be assigned to parameter `"name"` of type `"str"` in function `"greet"`

Needs `python.analysis.typeCheckingMode` set to `basic` — the default is `off`, which reports nothing.

The runtime doesn't care

```
>>> add("two", "three")  
'twothree'  
>>> add.__annotations__  
{ 'a': <class 'int'>, 'b': <class 'int'>, 'return': <class 'int'> }
```

- ▶ The hints are **right there** — and ignored
- ▶ The editor warned you; Python still ran it
- ▶ Hints are a **contract for humans and tools**, not a runtime guard

What you just saw

- ▶ A name resolved **L** → **E** → **G** → **B**, and a `NameError`'s cousin: `UnboundLocalError`
- ▶ `nonlocal` kept a `count` alive **between calls**
- ▶ Each `make_counter()` got its **own** independent counter
- ▶ Pylance flagged a bad call **before running** — once switched on
- ▶ Python ran the same bad call **happily**

Wrap-up

Q&A + this week's homework

Lab 5 — refactor into functions

1. `calculator.py` (Lab 2) → typed functions with docstrings:
`add`, `subtract`, `floor_divide`, `remainder`, `percentage`
2. `menu.py` (Lab 4) → a typed `dispatch(command: str) -> str`
3. Use a **default** and a **keyword-only** parameter where sensible
4. Every function: a **docstring** + full **type hints**
5. **Behaviour must not change** · `ruff check` stays clean

Mirror **Demo 9** for signatures, **Demo 10** for hints.

Questions?

- ▶ Why does `count = count + 1` fail but `print(count)` work?
- ▶ When *should* I use `global`?
- ▶ Do I have to annotate **everything**?
- ▶ Where do `*args` / `**kwargs` actually pay off?

Ask anything — this is the week to get scope straight.

Next week

- ▶ **Week 6 — Lists & Tuples**
- ▶ The containers your functions will **take in** and **hand back**
- ▶ `list[str]` stops being a hint and becomes a **thing you build**
- ▶ Your typed `dispatch()` starts managing **real tasks**

See you in the lab.

Questions?